The Julius Maximilian University of Würzburg

Introduction to Natural Language Processing
Mini Project 1

Student Names: Erik Seiferth, Alzbeta Hrabosova and Kerem Keptig
Student Numbers: 3161911, 3167903, 3168802
Date of Submission: 18 January 2025

Chapter 1

Introduction

In this report, we implement the Skip-Gram Word2Vec model with negative sampling using the `PyTorch` framework. The approach consists of:

1. Data preparation and vocabulary building from a text dataset ("text8" in our example).
2. Conversion of words to indices.
3. Generation of Skip-Gram pairs using a specified window size.
4. Calculation of a smoothed unigram distribution for negative sampling.
5. Defining and training the Skip-Gram neural network model.

1.1 Data Extraction and Preprocessing

1.1.1 Extracting Subset of Text

In large-scale data scenarios (such as "text8"), we can limit the dataset size to the first `N` words to save time and resources:

The data preparation process ensures efficiency by checking for existing subset files to avoid redundant work. If not found, the script unzips the `text8` dataset, selects the first `n` words, and saves them to a new file for use in downstream tasks like vocabulary building and training.

1.1.2 Building the Vocabulary

We then build a vocabulary of the most frequent words, reserving the special token `<UNK>` for out-of-vocabulary words.

```
def build_vocabulary(input_filename="text8_20m.txt", data_folder="data",
    vocab_size=60000):
    vocab_path = os.path.join(data_folder, "vocabulary.pkl")
    if os.path.exists(vocab_path):

        with open(vocab_path, "rb") as f:
            vocabulary = pickle.load(f)
```

```
        return vocabulary

    input_path = os.path.join(data_folder, input_filename)
    with open(input_path, "r") as f:
        text = f.read()
    words = text.split()

    word_counts = Counter(words)
    most_common = word_counts.most_common(vocab_size - 1)
    vocabulary = {word: count for word, count in most_common}

    vocabulary["<UNK>"] = sum(count for word, count in word_counts.items()
        if word not in vocabulary)

    with open(vocab_path, "wb") as f:
        pickle.dump(vocabulary, f)
    return vocabulary
```

- **Most common words identification:** We use `Counter` from the `collections` module to efficiently find and sort words by frequency. This helps us rank words and reserve only the top `vocab_size`.
- **<UNK> token usage:** Any word not in the top `vocab_size` is assigned to the special token <UNK>. The count of these words is aggregated as the frequency of <UNK>.
- **Serialization with `pickle`:** We store the vocabulary and related mappings as `pickle` files for faster loading in downstream steps.
- **Drawbacks of a hard cutoff:** A large number of unique words may each appear only once or very infrequently. For instance, if there are 30,000 distinct words with a frequency of 1, but your vocabulary cutoff only excludes 20,000, then the system must "arbitrarily" decide which 20,000 are mapped to <UNK> while the other 10,000 remain as separate tokens. This highlights a limitation: certain rare words may appear once but make it into the vocabulary, whereas other words with the same frequency might end up as <UNK>.
- **Potential improvements:** A more sophisticated approach could account for other factors (e.g., domain relevance, morphological analysis) rather than using a purely frequency-based cutoff. However, for simplicity and computational efficiency, a hard frequency cutoff is commonly used in many systems.

1.1.3 Assigning Indices to Words

Each word in the vocabulary requires a unique integer index. We create both `word_to_index` and `index_to_word` mappings:

```
def assign_indices(vocabulary, data_folder="data"):
    indices_path = os.path.join(data_folder, "word_indices.pkl")
    if os.path.exists(indices_path):
        with open(indices_path, "rb") as f:
            word_to_index, index_to_word = pickle.load(f)
        return word_to_index, index_to_word

```

```
    word_to_index = {word: idx for idx, (word, _) in
        enumerate(vocabulary.items())}
    index_to_word = {idx: word for word, idx in word_to_index.items()}

    with open(indices_path, "wb") as f:
        pickle.dump((word_to_index, index_to_word), f)
    return word_to_index, index_to_word
```

1.2 Converting Words to Indices and Generating Skip-Gram Pairs

We then read our text file, convert each word into its corresponding index, and produce (center, context) pairs for the Skip-Gram training.

1.2.1 Convert Text to Indices

```
def convert_to_indices(
    input_filename="text8_20m.txt",
    output_filename="text8_indices.txt",
    data_folder="data",
    word_to_index=None,
):
    indices_npy_path = os.path.join(data_folder, "text8_indices.npy")
    if os.path.exists(indices_npy_path):
        print(f"Indices file {indices_npy_path} already exists.")
        return

    input_path = os.path.join(data_folder, input_filename)
    with open(input_path, "r") as f:
        text = f.read()

    words = text.split()
    indices = [word_to_index.get(word, word_to_index["<UNK>"]) for word in
        words]
    np.save(indices_npy_path, np.array(indices))
```

1.2.2 Generate Skip-Gram Pairs

Skip-Gram pairs are formed by selecting a center word and its nearby context words:

```
def generate_skip_gram_pairs(data_folder="data", window_size=2,
    index_to_word=None):
    pairs_path = os.path.join(data_folder, "skip_gram_pairs.txt")
    indices_path = os.path.join(data_folder, "text8_indices.npy")

    if not os.path.exists(indices_path):
        raise FileNotFoundError("Indices file not found. Run
            convert_to_indices first.")

    indices = np.load(indices_path)
    skip_gram_pairs = []

    for i in range(len(indices)):
```

```
        center_word = indices[i]
        start = max(i - window_size, 0)
        end = min(i + window_size + 1, len(indices))
        for j in range(start, end):
            if j != i:
                context_word = indices[j]
                skip_gram_pairs.append((center_word, context_word))

    with open(pairs_path, "w") as f:
        for center, context in skip_gram_pairs:
            f.write(f"{center},{context}\n")
```

- `window_size` controls how many words on each side of the center are considered context.
- The pairs are simply written as `center,context` to a text file for use in training.

1.3 Negative Sampling Distribution

To perform negative sampling, we compute a smoothed unigram distribution $p(w)^{3/4}$:

```
    freqs = list(vocabulary.values())
    total_count = sum(freqs)
    unigram_distribution = {w: f/total_count for w,f in vocabulary.items()}

    alpha = 0.75
    smoothed_values = [p**alpha for p in unigram_distribution.values()]
    normalization_factor = sum(smoothed_values)
    smoothed_unigram_distribution = {
        w: (unigram_distribution[w]**alpha) / normalization_factor
        for w in vocabulary.keys()
    }
```

- **Raw Frequency Issue:** Using raw frequencies as negative sampling probabilities can overrepresent very frequent words, causing the model to learn less effectively from rare words.
- **Exponent α (Typically 0.75):** Applying $\alpha < 1$ (e.g., 0.75) shrinks the probability of highly frequent words and boosts the probability of less frequent words.
- **Re-normalization:** After raising each probability to the power of α, we sum them up to get a normalization factor. We then divide each smoothed value by this factor to ensure all probabilities still sum to 1.
- **Effect on Negative Sampling Efficiency:** By not always drawing the same high-frequency words as negative examples, the model sees a more diverse set of negative samples. This leads to more balanced training and often improves embedding quality.

1.4 Skip-Gram Model Implementation

We use a neural network model with two `Embedding` layers:

- `input_embeddings` for mapping center words to embeddings.
- `output_embeddings` for context words and negative samples.

```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size, embedding_dim=100):
        super(SkipGramModel, self).__init__()
        self.input_embeddings = nn.Embedding(vocab_size, embedding_dim)
        self.output_embeddings = nn.Embedding(vocab_size, embedding_dim)

        nn.init.uniform_(self.input_embeddings.weight,
            a=-0.5/embedding_dim, b=0.5/embedding_dim)
        nn.init.uniform_(self.output_embeddings.weight,
            a=-0.5/embedding_dim, b=0.5/embedding_dim)

    def forward(self, center_words, context_words, negative_words):
```

```
        center_embeds = self.input_embeddings(center_words)
        context_embeds = self.output_embeddings(context_words)
        negative_embeds = self.output_embeddings(negative_words)

        positive_score = torch.sum(center_embeds * context_embeds, dim=1)

        negative_score = torch.bmm(negative_embeds,
            center_embeds.unsqueeze(2)).squeeze(2)

        return positive_score, negative_score

    def loss(self, positive_score, negative_score):
        pos_loss = -torch.mean(log_sigmoid(positive_score))
        neg_loss = -torch.mean(torch.sum(log_sigmoid(-negative_score),
            dim=1))
        return pos_loss + neg_loss
```

1.4.1 Log-Sigmoid Helper

Since `F.logsigmoid` is deprecated, a custom `log_sigmoid` function is defined:

```
def log_sigmoid(x):
    return -F.softplus(-x)
```

1.5 Dataset and Dataloader

A custom `Dataset` is created to:

- Read stored skip-gram pairs.
- Sample negative words based on the smoothed distribution.

```
class SkipGramDataset(Dataset):
    def __init__(
        self,
        data_folder="data",
        word_to_index=None,
        smoothed_distribution=None,
        negative_samples=5,
        subset_fraction=1,
    ):
        pairs_file = os.path.join(data_folder, "skip_gram_pairs.txt")
        with open(pairs_file, "r") as f:
            lines = f.readlines()

        subset_size = int(len(lines) * subset_fraction)
        lines = lines[:subset_size]

        self.pairs = []
        for line in lines:
            center, context = line.strip().split(",")
            self.pairs.append((int(center), int(context)))

        words = list(smoothed_distribution.keys())
```

```
23          self.word_indices = [word_to_index[w] for w in words]
24          self.word_probs = torch.tensor([smoothed_distribution[w] for w in
                words], dtype=torch.float32)
25          self.negative_samples = negative_samples
26
27      def __len__(self):
28          return len(self.pairs)
29
30      def __getitem__(self, idx):
31          center, context = self.pairs[idx]
32
33          neg_indices = torch.multinomial(self.word_probs,
                self.negative_samples, replacement=True)
34          neg_samples = [self.word_indices[i] for i in neg_indices]
35          return center, context, torch.tensor(neg_samples, dtype=torch.long)
```

1.6 Training

Following sections demonstrate how to train and save the model:

- **Data Preparation:**
  The script begins by calling functions to extract a subset of words from the raw `text8` dataset, build a vocabulary, assign indices to each word, convert the text into an array of word indices, generate skip-gram pairs, and compute the negative sampling distribution. These steps ensure that the training data is properly organized before the model begins training.

- **Hyperparameter and Model Setup:**
  Important hyperparameters (like `embedding_dim`, `negative_samples`, `batch_size`, `epochs`, and the learning rate `lr`) are set. Then a `SkipGramDataset` is created and wrapped in a `DataLoader` to efficiently stream batches of data to the model during training. The `SkipGramModel` is instantiated with a vocabulary size and embedding dimension, and an `Adam` optimizer is set up with the specified learning rate.

- **Training Loop:**
  The script enters a loop over the specified number of epochs. For each epoch, it iterates through all batches in the `DataLoader`. In each iteration:

  1. It retrieves center words, context words, and negative samples.
  2. It zeros out the gradients (`optimizer.zero_grad()`).
  3. It performs a forward pass to compute `positive_score` and `negative_score`.
  4. It calculates the total loss by combining positive and negative losses.
  5. It backpropagates (`loss.backward()`) and updates the model parameters (optimizer.step()).

- **Monitoring Loss:**
  A running total of the loss is kept throughout each epoch. Periodically (every 10 steps in the code), the current loss is printed to provide insight into how training is progressing. After each epoch, the average loss for that epoch is computed and printed.

- **Model Saving and Best Model Tracking:**
  After each epoch completes, the model's state (i.e., its learned parameters) is saved to `skipgram_model.pth`. If the new average loss is lower than the previously recorded best loss, it updates `best_loss` and saves the new model with the best loss, which preserves the best-performing version of the model seen so far.

1.7 Results and Evaluation

1.7.1 Evaluation

- At first the word embeddings from the saved model file are loaded. The embeddings are stored in the `input_embeddings.weight` key of the model file, which corresponds to the learned word embeddings in the neural network.
- It also loads the vocabulary (a dictionary mapping words to indices) which is necessary to map words to their corresponding embedding vectors.
- For each word pair in the WordSim-353 dataset, the cosine similarity between the corresponding embeddings is computed.
- The human-assigned similarity scores for the pairs are collected as well, and the model's computed cosine similarities are compared with these scores.
- Spearman's rank correlation coefficient is calculated between the model's cosine similarities and the human scores. This statistic measures the strength and direction of the monotonic relationship between the two sets of values. A high positive correlation means that the model is doing well in predicting the human-assigned similarities.

1.7.2 Results

We trained the models using various parameter configurations, as summarized in Table Table 1.1. Unfortunately, due to limited computational resources and the constraints of our hardware, we were unable to perform additional experiments. The high computational cost, particularly with larger embedding sizes or increased numbers of epochs, posed significant challenges in terms of training time and resource usage. Despite these limitations, we achieved a maximum Spearman's rank correlation coefficient of 0.51 under the best-performing configuration. This setup utilized an embedding size of 150, a batch size of 254, a learning rate of 0.001, 5 epochs, and 7 negative samples.

Embedding size	**Batch size**	**Learning rate**	**Epochs**	**Negative samples**	**Spearman coef.**
150	128	0.001	5	7	0.5
150	128	0.001	2	8	0.44
200	64	0.001	2	10	0.41
150	254	0.001	5	7	**0.51**

Table 1.1: Training configurations and results

1.7.3 Visualization

- The words selected for visualization represent various categories, including: `king`, `queen`, `prince`, `princess`, `aunt`, `uncle`, `daughter`, `son`, `paris`, `france`, `london`, `england`, `apple`, `potato`, `mango`, `fruit`, `lion`, `wolf`, `tiger`, `elephant`, `car`, `truck`, `vehicle`, `bus`, `neptune`, `saturn`, `pluto`, `earth`
- The embeddings for these words were extracted, reduced to two dimensions using PCA, and plotted on a scatter plot with word labels.

The word embeddings visualization 1.1 shows clustering of words based on their semantic categories. For instance, words related to "cities and countries" are grouped together, as are words related to "humans." However, some inconsistencies are observed; for example, words from the "fruits" category are not closely clustered and mixed with words belonging to "planets", indicating the model's difficulty in assigning these words to a coherent context.

Several factors may contribute to this behavior. One potential reason is the limited training data, as only a subset of 20 million words from the full dataset was used. Additionally, learning word embeddings that effectively capture semantic relationships is a highly complex NLP task, making it challenging to achieve optimal results.

Despite these limitations and a Spearman's rank correlation coefficient of 0.51, the model successfully learned meaningful word embeddings.

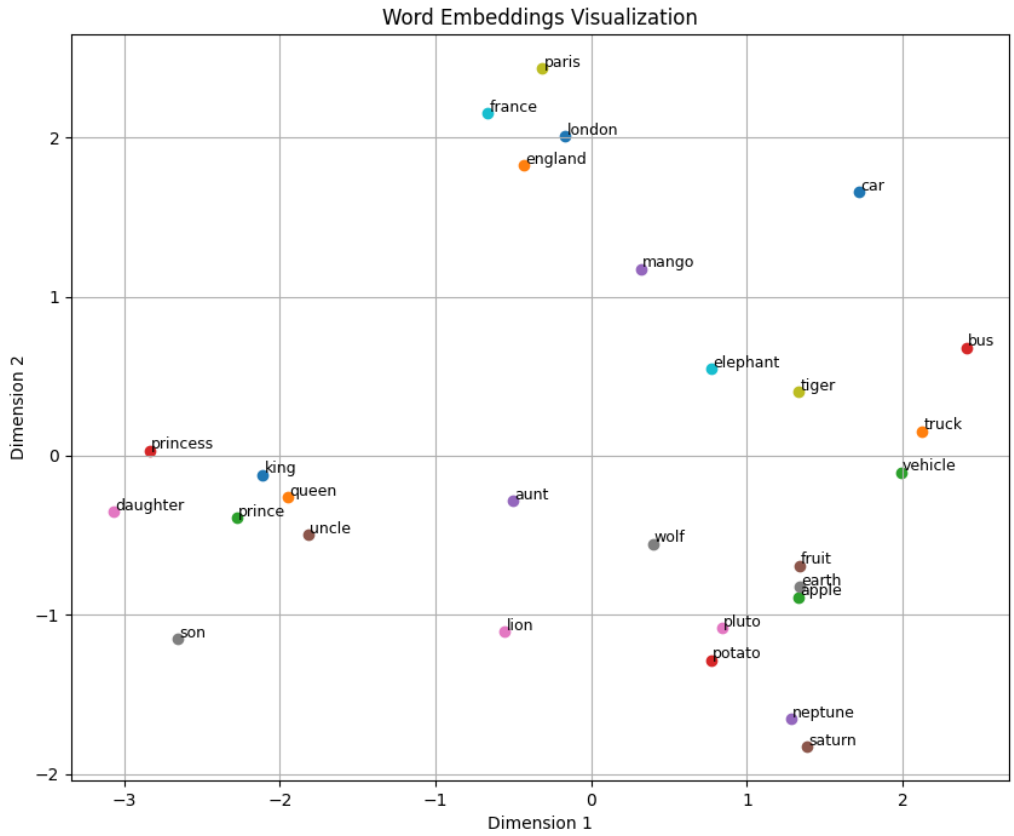

Figure 1.1: PCA visualization of word embeddings

1.8 Conclusions

This report presents the implementation and evaluation of a Skip-Gram, model with negative sampling using the `PyTorch` framework. Through a detailed exploration of data preparation, model architecture, training process, and evaluation, the project demonstrated the potential of neural embeddings to capture semantic relationships between words.

Despite the constraints of computational resources and the use of a limited subset of the dataset, the model achieved a Spearman's rank correlation coefficient of 0.51, demonstrating its ability to learn meaningful word embeddings. Visualization of these embeddings further confirmed the model's effectiveness in grouping semantically similar words, though some inconsistencies were observed, particularly in less distinct categories like "fruits" and "planets."

Overall, the results underscore the utility of the Skip-Gram model in NLP tasks while highlighting the importance of computational resources, hyperparameter tuning, and dataset size for optimizing performance. Future work could focus on scaling the dataset, experimenting with advanced embedding techniques, and further optimizing hyperparameters to achieve better semantic representations.